



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика, искусственный интеллект и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа №8
по курсу «Языки и методы программирования»
«Разработка шаблона класса»

Студент группы ИУ9-21Б Яннаев А. С.

Преподаватель Посевин Д. П.

Москва 2025

1 Цель работы

Целью данной работы является изучение шаблонов классов языка C++.

2 Задание

Согласно выбранному из таблиц 1–16 описанию требуется составить шаблон класса, разместив его в отдельном заголовочном файле. Проверку работоспособности класса требуется организовать в функции `main`, размещённой в файле «`main.cpp`».

3 Результаты

Исходный код программы представлен в 1, 2, 3, 4.

Листинг 1: Файл

```
1 #include <iostream>
2 #include "polygon.hpp"
3
4 int main() {
5     // Test Polygon class with triangle
6     Polygon<double, 3> triangle;
7     triangle.addVertex(0, 0);
8     triangle.addVertex(3, 0);
9     triangle.addVertex(0, 4);
10
11     std::cout << "triangle perimeter: " << triangle.perimeter() << std::
endl;
12     std::cout << "triangle area: " << triangle.area() << std::endl;
13
14     // Test adding vertex to create quadrilateral
15     auto quad = triangle.addVertexNew(3, 4);
16     std::cout << "quadrilateral perimeter: " << quad.perimeter() << std
::endl;
17
18     // Test with another polygon (pentagon)
19     Polygon<double, 5> pentagon;
20     pentagon.addVertex(0, 0);
21     pentagon.addVertex(2, 0);
22     pentagon.addVertex(3, 1);
23     pentagon.addVertex(2, 2);
```

```

24     pentagon.addVertex(0, 2);
25     std::cout << "pentagon perimeter: " << pentagon.perimeter() << std::
endl;
26
27     return 0;
28 }

```

Листинг 2: Файл

```

1 #ifndef POLYGON_HPP
2 #define POLYGON_HPP
3
4 #include <vector>
5 #include <cmath>
6 #include <stdexcept>
7
8 template <typename T, int N>
9 class Polygon {
10 private:
11     std::vector<std::pair<T, T>> vertices;
12
13 public:
14     Polygon() {
15         static_assert(N >= 3, "polygon must have at least 3 vertices");
16     }
17
18     void addVertex(T x, T y) {
19         vertices.emplace_back(x, y);
20     }
21
22     double perimeter() const {
23         if (vertices.size() != N) {
24             throw std::runtime_error("incorrect number of vertices");
25         }
26
27         double p = 0.0;
28         for (int i = 0; i < N; ++i) {
29             int j = (i + 1) % N;
30             double dx = vertices[j].first - vertices[i].first;
31             double dy = vertices[j].second - vertices[i].second;
32             p += std::sqrt(dx*dx + dy*dy);
33         }
34         return p;
35     }
36
37     template<int M = N>
38     typename std::enable_if<M == 3, double>::type area() const {

```

```

39         if (vertices.size() != 3) {
40             throw std::runtime_error("incorrect number of vertices for
triangle");
41         }
42
43         // corrected distance calculations:
44         double a = std::sqrt(
45             std::pow(vertices[1].first - vertices[0].first, 2) +
46             std::pow(vertices[1].second - vertices[0].second, 2)
47         );
48         double b = std::sqrt(
49             std::pow(vertices[2].first - vertices[1].first, 2) +
50             std::pow(vertices[2].second - vertices[1].second, 2)
51         );
52         double c = std::sqrt(
53             std::pow(vertices[0].first - vertices[2].first, 2) +
54             std::pow(vertices[0].second - vertices[2].second, 2)
55         );
56         double s = (a + b + c) / 2;
57         return std::sqrt(s * (s - a) * (s - b) * (s - c));
58     }
59
60     Polygon<T, N+1> addVertexNew(T x, T y) const {
61         Polygon<T, N+1> newPoly;
62         for (const auto& v : vertices) {
63             newPoly.addVertex(v.first, v.second);
64         }
65         newPoly.addVertex(x, y);
66         return newPoly;
67     }
68 };
69
70 #endif

```

Листинг 3: Файл

```

1 #include <iostream>
2 #include "stack.hpp"
3
4 int main() {
5     // Test Stack with integers
6     Stack<int> intStack;
7     intStack.push(3);
8     intStack.push(1);
9     intStack.push(4);
10    std::cout << "max in int stack: " << intStack.max() << std::endl;
11    intStack.pop();

```

```

12     std::cout << "new top after pop: " << intStack.top() << std::endl;
13
14     // Test Stack with strings
15     Stack<std::string> strStack;
16     strStack.push("123");
17     strStack.push("456");
18     strStack.push("789");
19     std::cout << "\nmax in string stack: " << strStack.max() << std::
endl;
20
21     // Test string reversal
22     strStack.reverseStrings();
23     std::cout << "top after reverse: " << strStack.top() << std::endl;
24     strStack.pop();
25     std::cout << "next after pop: " << strStack.top() << std::endl;
26
27     return 0;
28 }

```

Листинг 4: Файл

```

1  #ifndef STACK_HPP
2  #define STACK_HPP
3
4  #include <stack>
5  #include <string>
6  #include <algorithm>
7  #include <stdexcept>
8
9  template <typename T>
10 class Stack {
11 private:
12     std::stack<T> data;
13     std::stack<T> maxStack; // auxiliary stack for tracking max element
14
15 public:
16     // push element to stack
17     void push(const T& value) {
18         data.push(value);
19         if (maxStack.empty() || value >= maxStack.top()) {
20             maxStack.push(value);
21         }
22     }
23
24     // pop element from stack
25     void pop() {
26         if (data.empty()) {

```

```

27         throw std::runtime_error("stack is empty");
28     }
29     if (data.top() == maxStack.top()) {
30         maxStack.pop();
31     }
32     data.pop();
33 }
34
35 // get top element
36 T top() const {
37     if (data.empty()) {
38         throw std::runtime_error("Stack is empty");
39     }
40     return data.top();
41 }
42
43 // check if stack is empty
44 bool empty() const {
45     return data.empty();
46 }
47
48 // get max element in constant time
49 T max() const {
50     if (maxStack.empty()) {
51         throw std::runtime_error("stack is empty");
52     }
53     return maxStack.top();
54 }
55 };
56
57 // specialization for Stack<string>
58 template <
59 class Stack<std::string> {
60 private:
61     std::stack<std::string> data;
62     std::stack<std::string> maxStack;
63
64 public:
65     void push(const std::string& value) {
66         data.push(value);
67         if (maxStack.empty() || value >= maxStack.top()) {
68             maxStack.push(value);
69         }
70     }
71
72     void pop() {

```

```

73         if (data.empty()) {
74             throw std::runtime_error("stack is empty");
75         }
76         if (data.top() == maxStack.top()) {
77             maxStack.pop();
78         }
79         data.pop();
80     }
81
82     std::string top() const {
83         if (data.empty()) {
84             throw std::runtime_error("stack is empty");
85         }
86         return data.top();
87     }
88
89     bool empty() const {
90         return data.empty();
91     }
92
93     std::string max() const {
94         if (maxStack.empty()) {
95             throw std::runtime_error("stack is empty");
96         }
97         return maxStack.top();
98     }
99
100    // reverse all strings in stack (constant time operation)
101    void reverseStrings() {
102        std::stack<std::string> temp;
103        while (!data.empty()) {
104            std::string s = data.top();
105            std::reverse(s.begin(), s.end());
106            temp.push(s);
107            data.pop();
108        }
109        data = std::move(temp);
110    }
111 };
112
113 #endif

```

Результат запуска представлен на рисунке 1.

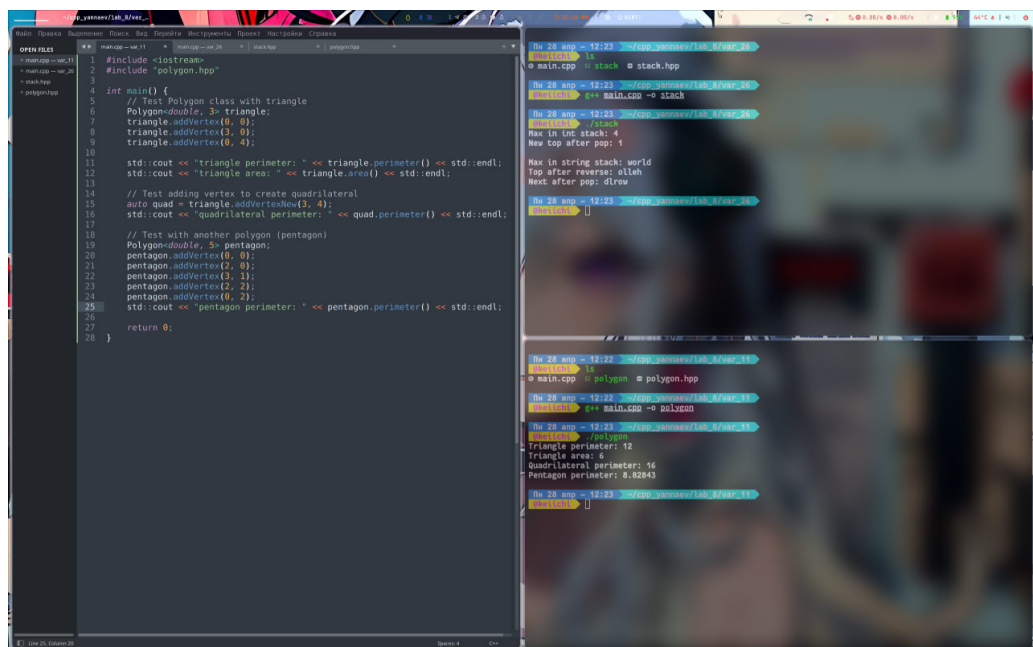


Рис. 1 — Результат